
Leant

a GHCi-style REPL for Lean 4

an overview and tutorial, with a detailed tour of `:synth`

An **experimental** tool in active development — interfaces, output, and capabilities change frequently.

Part of the *ProveIt* repository, `Tools/Leant`. July 2026.

Contents

1	Introduction	2
2	Getting started	2
3	The commands	3
3.1	Session and evaluation	3
3.2	Discovery	3
3.3	Proving and synthesis	3
3.4	Persistence and utility	4
4	Interactive proving	4
5	Term synthesis: :synth	4
5.1	Higher-order plumbing	5
5.2	Impossibility, proved	6
5.3	Inductive types	6
5.4	Classical candidates	7
5.5	Dependent formulas as cargo	8
5.6	Synthesis inside a proof	8
5.7	Engines, budgets, and honesty about both	9
5.8	What :synth will not do	9
6	How it works, briefly	10
7	Status	10

1 Introduction

Leant is an interactive read-eval-print loop for Lean 4, in the spirit of GHCi: you type expressions and they are evaluated, you type declarations and they enter the session, and a family of colon-commands gives you type queries, documentation, search, interactive proving, and — the subject of half of this document — automatic *term synthesis* with machine-checked answers.

Lean 4 itself is normally driven from an editor: the language server shows goals and diagnostics as you edit a file. Leant complements that workflow with a conversational one. It is aimed at exploration — trying a lemma, poking at a definition, asking *is this type even inhabited?* — where the unit of work is a line, not a file.

experimental

Leant is an **experimental project in active development**. It is a research vehicle, not a supported product: commands appear, change shape, and disappear between commits; output formats are not stable; and some features are deliberately bounded approximations. Every transcript in this document was produced by the tool at the date on the title page and may not match the tool you build tomorrow. When this document and `:help` disagree, believe `:help`.

Two implementations live side by side in the repository: `Tools/LeantPy` (the Python original, whose backend management is delegated to `LeanInteract`) and `Tools/Leant` (the Haskell port, now the primary implementation, which speaks the backend protocol directly). They share the command set and the same session semantics; term synthesis (§5) is Haskell-only, because it embeds the Djex synthesis library in-process. Everything below describes the Haskell implementation.

Under the hood, Leant drives the community `leanprover-community/repl` binary over a JSON protocol: every input becomes a command against a persistent Lean environment, and every claim Leant makes — including every synthesized term — is checked by that backend before it is shown. Sessions survive backend crashes (the session history replays automatically), can be saved and restored (`:pickle/:unpickle`), and can be recorded as transcripts.

2 Getting started

Build the binary once with `cabal build exe:leant` (GHC 9.12.4; the vendored `lib/Djex` submodule must be initialized), then run `leant.cmd` — or the built executable directly. Leant finds the Lean backend the same way `LeanInteract`'s cache laid it out, or accepts an explicit `--repl-exe/LEANT_BACKEND`. Without a Lake project it runs against the plain Lean 4 standard library, which is also the least memorable thing about it: no imports, subsecond startup.

A session begins as a calculator and escalates quietly:

session

```
λ> 2 + 2
4
λ> it * 10
```

```

40
λ> def double (n : Nat) : Nat := n + n
λ> double 21
42
λ> :type double
double : Nat → Nat

```

Expressions are tried with `#eval` and fall back to `#check`; the last evaluated value is bound as `it`, GHCi-style. Declarations (`def`, `theorem`, `inductive`, ...) extend the session environment, and any `#`-command (`#eval`, `#check`, `#print axioms`) passes straight through. Multi-line input opens automatically when a line is syntactically incomplete — type a structure header and keep going — and an empty line submits it.

3 The commands

3.1 Session and evaluation

<code>:help, :h, :?</code>	show the command summary
<code>:quit, :q</code>	exit
<code>:type EXPR, :t</code>	show the type of EXPR (<code>#check</code>)
<code>:info NAME, :i</code>	show the definition of NAME (<code>#print</code>)
<code>:load FILE, :l</code>	reset the session and load a <code>.lean</code> file
<code>:reload, :r</code>	reload the last loaded file
<code>:import MOD</code>	add an import (rebuilds the session)
<code>:imports</code>	list active imports
<code>:undo</code>	revert the last state-changing command
<code>:reset</code>	clear all definitions (keeps imports)
<code>:history</code>	show the session's state-changing commands
<code>:set OPT VAL</code>	a Lean <code>set_option</code> that persists in the session

3.2 Discovery

<code>:browse [NS]</code>	list declarations in a namespace, or the session's own
<code>:browse! NS</code>	...including compiler-generated auxiliaries
<code>:doc NAME</code>	show a docstring
<code>:search TEXT</code>	search declaration names, case-insensitively
<code>:search? TYPE</code>	proof search: what proves TYPE? (via <code>exact?</code>)

3.3 Proving and synthesis

<code>:prove [PROP]</code>	prove PROP interactively, tactic by tactic (without an argument: resume the last sorry)
<code>:synth TYPE</code>	synthesize verified terms of TYPE; candidates are bound as <code>it1</code> (= <code>it</code>), <code>it2</code> , ...(<code>\$5</code>)

```

:set synth-engine E      djinn (default) | exference | both
:set synth-steps N       Exference's step budget (default 4096)
:set synth-classical B   offer classical candidates for constructively refuted goals (on/off, default on)

```

3.4 Persistence and utility

```

:transcript [F|on|off]  record a transcript of the session
:timestamps [on|off]    timestamp transcript entries
:pickle FILE            save the environment to FILE (.olean)
:unpickle FILE          restore an environment
:env                    show the backend environment id
:time                   toggle per-command timing
:~ COMMAND              run a shell command

```

4 Interactive proving

:prove turns the prompt into a tactic loop against the backend's proof-state protocol. Every line is a tactic; :undo pops one step; :qed NAME assembles the accumulated script into a theorem and replays it into the session, so the result is usable afterwards.

session

```

λ> :prove ∀ p q : Prop, (p → q) → p → q ∧ p
├ ∀ (p q : Prop), (p → q) → p → q ∧ p
├> intro p q h hp
p q : Prop
h : p → q
hp : p
├ q ∧ p
├> exact ⟨h hp, hp⟩
All goals accomplished
├> :qed mp_and
saved: theorem mp_and : ∀ p q : Prop, (p → q) → p → q ∧ p

```

A sorry in an ordinary declaration prints its goal and offers the same loop via bare :prove. Prove mode also cooperates with :synth, which is where it gets interesting (§5.6).

5 Term synthesis: :synth

:synth TYPE answers the question “*write me a term of this type*” — or, on the propositions-as-types side of the mirror, “*prove this*” — and sometimes the stronger question “*show me that no such term exists*.” It is built on the vendored Djex library, linked in-process. Djex began by merging two classic Haskell program synthesizers behind one checked synthesis foundation — **Djinn**, a complete and terminating proof search for intuitionistic propositional logic (Dyckhoff’s LJ_T calculus) that emits programs, and **Exference**, a ranked heuristic search under explicit budgets — and has since moved well beyond both originals. It fixes a substantial

number of their bugs (notably verdict honesty: an exhausted search over an approximated space is reported as *no evidence*, never as a refutation); it implements markedly better type-class support, with a unified, validated constraint contract shared by both engines — Exference resolves givens, superclasses, and instances, Djinn checks contexts and synthesizes dictionary-independent terms; and it adds a growing level of support for rank-N and impredicative types: goal-side quantifiers open through polarity-aware plan families, quantified hypotheses are instantiated at a bounded set of sequent-supplied types, and impredicative instantiation is admitted under a guard that never invents a polytype the query did not supply. That scope is still being improved; Leant consumes Djex as a library behind a deliberately narrow engine boundary, so each improvement arrives by version bump. The engines speak Haskell types; Leant translates Lean goals into their fragment and their answers back into Lean.

Three design rules keep this honest:

- **The engine is never trusted.** Every candidate is re-elaborated by the Lean backend against the exact goal (example : (T) := term) before you see it. A rendering bug or an engine bug costs a dropped candidate, never a wrong answer.
- **Refusals are honest.** Goals outside the supported fragment are refused with a reason, not silently mangled.
- **Negative verdicts are labeled by strength.** “Provably uninhabited” appears only when the translation was complete and lossless; anything weaker is reported as “no term found within bounds,” which claims nothing.

5.1 Higher-order plumbing

The sweet spot is the structural fragment: arrows, products, sums, $\perp$, $\top$, negation, *Iff*, and quantifiers over opaque variables — the “plumbing” terms one writes constantly. Free capital identifiers are auto-bound, so quick queries stay quick:

session

```
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1 fun f g x => f x (g x)
λ> :synth (A × (B ⊕ C) → (A × B) ⊕ (A × C))
  it1 fun (x, y) => match y with | .inl z => .inl (x, z) | .inr w => .inr (x, w)
λ> :synth (∀ p q : Prop, (p ↔ q) → ¬p → ¬q)
  it1 fun _ _ (f) k x => k (f x)
```

The first is the *S* combinator; the second distributes a product over a sum; the third is contraposition across an *Iff*, with the backward implication projected out by the pattern $(_, f)$. Binders are named by role — functions *f g h*, values *x y z*, negations and continuations *k* — which is a small thing that makes large candidates legible.

Candidates are ranked smallest-first and *bound into the session* as *it1*, *it2*, ..., with bare *it* the best one. They are ordinary definitions; you can evaluate them immediately:

session

```
λ> :synth (a → a → a)
  it1 fun _ x => x
  it2 fun x _ => x
λ> #eval it2 "left" "right"
"left"
```

Both inhabitants of $a \rightarrow a \rightarrow a$ that matter, and picking one is just using its name. For *programs* — as opposed to proof-irrelevant propositions — candidate choice is the whole point.

5.2 Impossibility, proved

When Djinn’s complete search exhausts a fully translated goal, failure is a theorem:

session

```
λ> :synth (∀ a b : Type, a → b)
provably uninhabited – no closed term of this polymorphic type exists
```

No Lean tactic gives you this answer (a failing `exact?` proves nothing). Note the careful wording: the verdict is about *closed terms of the polymorphic type*. It does not say any particular instance $a \rightarrow b$ is empty — choose $a = b$ and it isn’t — and for propositions it is about *constructive* provability, a distinction §5.4 exploits. When the translation had to hide structure behind an opaque atom, the verdict honestly downgrades:

session

```
λ> :synth (∀ a : Type, Sigma (fun _ : Nat => a))
no term found within bounds (opaque atoms `Nat` hide structure the engine cannot
↳ analyze – not a refutation)
```

5.3 Inductive types

A non-recursive, non-indexed inductive or structure applied to all its parameters is expanded into a generalized sum of products: constructors become introduction rules, case analysis the elimination rule. This covers the standard library’s little workhorses —

session

```
λ> :synth Ordering
  it1 .lt
  it2 .eq
  it3 .gt
λ> :synth (∀ a : Type, Option (Option a) → Option a)
  it1 fun _ _ => .none
  it2 fun _ x => match x with | .none => (.none) | .some y => y
λ> :synth (∀ e a b : Type, Except e a → (a → b) → Except e b)
  it1 fun _ _ x f => match x with | .error y => .error y | .ok z => .ok (f z)
```

— `Option.join` and `Except.map` synthesized rather than remembered. It extends to Type-valued classes-as-data like `Decidable`, where the eliminations write themselves:

session

```
λ> :synth (∀ p : Prop, Decidable p → Decidable (¬ p))
  it1 fun _ x => match x with | .isFalse k => .isTrue k | .isTrue y => .isFalse (fun
    ↪ k1 => k1 y)
```

Session-declared types participate the moment you declare them, and refutations over expanded inductives remain sound, because the engine saw the complete constructor list:

session

```
λ> structure Pair (A B : Type) where
...>   fst : A
...>   snd : B
...>
λ> :synth (∀ a b : Type, a → b → Pair a b)
  it1 fun _ _ x y => ⟨x, y⟩
λ> :synth (∀ a b : Type, Pair a b → Empty)
  provably uninhabited — no closed term of this polymorphic type exists
```

Recursive types cannot be expanded — elimination is where undecidability lives — but their *constructors* are still sound introduction rules, so building is possible even where analysis is not:

session

```
λ> :synth Nat
  it1 Nat.zero
  it2 Nat.succ Nat.zero
λ> :synth (∀ a : Type, a → List a)
  it1 fun _ _ => List.nil
```

5.4 Classical candidates

Peirce’s law has no constructive inhabitant, and `:synth` can prove that. But rather than stopping at the refutation, it then asks the classical question, and answers with a term whose spelling announces exactly what was used:

session

```
λ> :synth (∀ p q : Prop, ((p → q) → p) → p)
  it1 fun _ _ f => match Classical.em _ with | .inl x => x | .inr k => f (fun y =>
    ↪ absurd y k)
λ> :synth (∀ p : Prop, ¬¬p → p)
  it1 fun _ k => match Classical.em _ with | .inl x => x | .inr k1 => absurd k1 k
```

These are the proofs a human would write: one case split on excluded middle, one absurd where the contradiction lands. Even statements that are classically true but intuitionistically unprovable in a less famous way come out as case splits — the Gödel–Dummett formula, for instance:

session

```

λ> :synth (∀ p q : Prop, (p → q) ∨ (q → p))
it1 fun _ => match Classical.em _ with | .inl x => (match Classical.em _ with |
  ↳ .inl _ => .inr (fun _ => x) | .inr k => .inr (fun y => absurd y k)) | .inr k1 =>
  ↳ .inl (fun z => absurd z k1)

```

Two searches stand behind this. First, one excluded-middle assumption per atomic subformula is handed to the intuitionistic engine — complete for the propositional fragment, since intuitionistic logic plus atom instances of $p \vee \neg p$ is classical propositional logic — and rendered as `Classical.em` case splits. If that misses, the goal’s double-negation translation runs instead (complete by Glivenko’s theorem) and the candidate is wrapped in `Classical.byContradiction`. Both kinds are verified like any other candidate. The whole behavior sits behind a switch:

session

```

λ> :set synth-classical off
synth classical: off
λ> :synth (∀ p : Prop, p ∨ ¬ p)
provably uninhabited — no closed term of this polymorphic type exists
(constructively — a classical proof may still exist; this is not a disproof of the
  ↳ proposition)

```

5.5 Dependent formulas as cargo

Leant’s translation deliberately does not analyze dependent types. But it does not refuse them either: a dependent subformula becomes an opaque *atom*, compared up to α -equivalence, which the engine can carry from one side of a goal to the other. Transport works; analysis does not:

session

```

λ> opaque P : Nat → Prop
λ> opaque Q : Prop
λ> :synth ((∀ n : Nat, P n) ∧ Q → Q ∧ (∀ n : Nat, P n))
it1 fun ⟨x, y⟩ => ⟨y, x⟩

```

The engine never looked inside $\forall n, P n$ — it swapped a sealed box. A goal that would require opening the box (an induction, a rewrite, a case split on an index) is refused with a reason; that work belongs to `:prove`.

5.6 Synthesis inside a proof

Bare `:synth` in prove mode targets the current goal *with its hypotheses as premises*. Candidates are offered as `it1`, `it2`, ... exactly as in the session, except that here `itN` splices the candidate applied to your hypotheses — so exact `it1` closes the goal:

session

```

λ> :prove ∀ p q : Prop, (p → q) → p → q ∧ p
├ ∀ (p q : Prop), (p → q) → p → q ∧ p
├> intro p q h hp
p q : Prop
h : p → q
hp : p
├ q ∧ p
├> :synth
(synthesizing with hypotheses p q h hp as premises)
  it1 fun _ _ f x => ⟨f x, x⟩
├> exact it1
All goals accomplished
├> :qed mp_and
saved: theorem mp_and : ∀ p q : Prop, (p → q) → p → q ∧ p

```

Unlike `exact?`, which finds an *existing* lemma, this *composes a new term* from the goal’s own material — a constructive complement to the finisher tactics, needing no premise database and no imports.

5.7 Engines, budgets, and honesty about both

The default engine is Djinn: complete, terminating, and the only source of refutations. `:set synth-engine exference` switches to the ranked heuristic search, and `both` runs the two together — Djinn’s candidates first, Exference’s new ones appended, negative verdicts only from the engine entitled to them. Exference runs under explicit budgets (`:set synth-steps`, default 4096), and truncation is always reported:

session

```

λ> :set synth-engine both
synth engine: both
λ> :synth ((A → B → C) → (A → B) → A → C)
  it1 fun f g x => f x (g x)
note: exference: search truncated: step limit reached, queue limit pruned 31990

```

The pure searches answer in microseconds to milliseconds; the cost center is backend verification, a few hundred milliseconds per candidate. A wall-clock guard (default 20 s, `LEANT_SYNTHTIMEOUT`) covers the quantified goals whose bounded instantiation can widen the space; hitting it is reported as “no answer,” never as a verdict.

5.8 What `:synth` will not do

Present limits, stated plainly: it will not synthesize recursion or induction (recursive types contribute constructors only); it will not analyze dependent structure (atoms are transported, never opened); quantifier handling beyond the propositional fragment follows Djex’s bounded rank-N and guarded impredicativity rules — sound and terminating over an undecidable space, and widening as Djex improves — so some higher-rank goals answer “no term found within bounds” rather than anything stronger; and although Djex itself resolves type-class evidence,

Leant currently leaves instance arguments to Lean’s elaborator during verification, which is better placed to resolve them. The fragment is bounded on purpose: inside it, answers are fast, verified, and — for refutations — final.

6 How it works, briefly

A `:synth` query passes through five checked stages. A Lean metaprogram (compiled once into a cached side environment) elaborates the goal and serializes it into the engines’ fragment: structural connectives are decomposed; qualifying inductives expand into constructor lists; everything else becomes an α -normalized opaque atom. The fragment translator either accepts the result or refuses with a reason. The selected engine searches; its candidates are rendered back into Lean syntax — constructor names restored, binders named by role, anonymous-constructor and leading-dot spellings preferred with fully qualified fallbacks. Finally the backend re-elaborates each candidate against the original goal, and only survivors are shown and bound. The engine boundary is narrow by design: the searches behind it are swappable, and nothing they emit is believed until Lean has type-checked it.

7 Status

Leant is a working prototype under active development inside the *ProveIt* repository. The Haskell implementation (`Tools/Leant`) is primary; the Python original (`Tools/LeantPy`) remains as a lighter alternative without synthesis. The synthesis design and its phased history are documented in `Tools/Leant/SYNTHESIS_PROPOSAL.md`; transcript-level golden tests live in `Tools/Leant/test`. Expect sharp edges: the project exists to find out how far a conversational, verification-gated synthesis workflow can be pushed, and it moves at the speed of that question.